Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 11/10/2026**

A Mini Project Report on

EventEase

A College Event Discovery and Booking Platform Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course
Agile Software Development & DevOps Lab
Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Suryansh Kalchuri	2401039
Krish Kanal	2401040
Miheeka Kewalramani	2401045

Under the guidance of

Dr. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

CERTIFICATE

This is to certify that the mini project entitled “EventEase: A College Event Discovery and Booking Platform Delivered Through an Agile and DevOps Pipeline” is a bona fide work of **Suryansh Kalchuri (2401039)**, **Krish Kanal (2401040)** and **Miheeka Kewalramani (2401045)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, at Thadomal Shahani Engineering College during the academic year 2026–2027.

_____ _____

Project Guide
Dr. Sanober Shaikh

Head of Department

Date: _____ Place: _____

ABSTRACT

Events in a college are announced through posters, group chats and word of mouth. Students miss events because the information is scattered, organisers cannot tell how many seats are left, and bookings are collected on paper or in shared forms where the same seat can be promised twice. EventEase addresses this with a web application in which a student browses the events of every club in one place, searches and filters them by category, registers, and books between one and ten tickets for an event, while an administrator adds events and sees how many seats of each event have been booked.

The application was used as the vehicle for applying the experiments of the Agile Software Development & DevOps Lab to a single product. The work was planned in Jira as epics and stories in two sprints (1 epic, 7 stories, 33 story points). Source code was kept in a GitHub repository, where the redesign of the website and the DevOps files were developed on separate branches and merged into `main` through pull requests, with the Jira key in every commit message. The application is a Flask service backed by an SQLite database. It is packaged as a Docker image, run by Docker Compose with a health check and a persistent data volume, and exposes a `/health` endpoint that reports whether the service is up and the database connected. A GitHub Actions workflow runs the tests on every push and pull request, and a Jenkins pipeline checks out `main`, runs the pytest suite with a JUnit report, builds the image, deploys the stack and finishes with a smoke test of the health endpoint. Nagios Core then checks the web portal, the health endpoint, the events page and the disk space.

The delivered system comprises nine college events from six clubs and committees, 21 automated tests that exercise registration, login, booking, capacity limits, admin access and the health check, a Jenkins run that ended in SUCCESS, and Nagios service checks that were all in the OK state. The report documents each stage with the team's own screenshots and explains how code that is committed to Git reaches a running and monitored deployment without manual steps in between.

Keywords: Agile, Jira, Git, GitHub Actions, Docker, Docker Compose, Jenkins, CI/CD, Nagios, Flask, SQLite, pytest

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Jira)
3	System Design
4	Application Implementation
5	DevOps Implementation (Git, Docker, Jenkins, Nagios)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

A college runs many events in a single semester: sports days, workshops, hackathons and community drives, each organised by a different student club or committee. Every organiser announces their event in their own way, so a student who wants to know what is happening this month has to check several places. On the organiser's side, the number of seats that are still free is usually unknown until the day of the event, and a booking made on paper or in a spreadsheet can be lost or counted twice.

The Agile Software Development & DevOps Lab introduced, through separate experiments, the practices used to deliver software reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. This mini project applies those practices to one realistic product, an event booking platform for the college, so that the way the tools work together becomes visible instead of each tool being seen on its own.

1.2 Problem Statement

To design, build and operate a web application that lets students discover college events and book tickets with live seat validation, and lets an administrator add events and watch bookings, and to deliver it through an Agile, automated pipeline in which every piece of work is tracked in Jira, versioned in Git, tested by GitHub Actions and Jenkins, packaged as a Docker container and observed by Nagios.

1.3 Objectives

1. Show all upcoming events of all clubs in one place, with search and category filters.
2. Let a student register, log in and book one to ten tickets, rejecting a booking that exceeds the seats left.
3. Show each student a personal page of bookings with ticket counts and totals.
4. Give an administrator a protected dashboard to add events and see booked seats against capacity.
5. Plan the work in Jira with epics, stories, story points, a sprint board and a completed-sprint report.
6. Keep all source code in GitHub, working on branches and merging through pull requests with the Jira key in each commit.
7. Containerise the application and run it with a single Docker Compose command, with a health check and a persistent data volume.
8. Build a Jenkins pipeline that checks out the code, runs the automated tests, builds the image, deploys the stack and smoke-tests the result.

9. Monitor the availability of the web portal, the health endpoint, the events page and the disk with Nagios.

1.4 Scope

The system serves one college and is loaded with nine events from six clubs and committees (Student Council, ISTE, CSI, RCTSEC, CodeTantra and NSS). Online payment, ticket cancellation and e-mail confirmation are outside the scope of this version; the ticket price is displayed and the total is calculated for information. The deployment target is a single Windows machine running Docker Desktop; the same Compose file would apply to a college server.

1.5 Mapping of Lab Experiments to the Project

Topic	Application in EventEase	Section
Version control with Git	GitHub repository, redesign and DevOps branches, merged pull requests, Jira key in commit messages	5.2
Containerisation with Docker	Dockerfile with a non-root user and a Gunicorn start command	5.3
Docker Compose	Service `eventease` with health check, data volume and private network	5.3
Jenkins CI	Jenkinsfile: checkout, pytest unit tests with a JUnit report	5.4
Automated deployment	Docker build, `docker compose up`, smoke test of `/health`	5.4
Agile project lifecycle (Jira)	Jira space with epics, stories and one sprint	2
Monitoring with Nagios	Nagios Core with one host and four service checks	5.5

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Student	Find events, book tickets and see own bookings	Home, Events, Event page, My Bookings, Register, Login
Administrator	Add events and monitor how many seats are booked	Admin dashboard, Add event
Development team	Plan, build, test, deploy and monitor the system	Jira, GitHub, Jenkins, Docker Desktop, Nagios

Table 2.1: User roles

2.2 Functional Requirements

- The home page shows a hero section, the number of upcoming events, the number of clubs, the number of free events and a card for every event.
- The events page lists all events as cards, with a search box (by event or club) and filter buttons for All and for each category.
- An event page shows the date, venue, price, seats left out of capacity and a description, with a booking box on the side.
- A visitor can register with a name, e-mail address and password of at least eight characters; the password is stored as a hash and a duplicate e-mail is rejected.
- A logged-in student can book 1 to 10 tickets for an event; a booking that would exceed the capacity is rejected, and a sold-out event shows a message instead of the form.
- My Bookings lists the student's bookings with the poster, date, venue, tickets, price per ticket and total.
- Only a user with the admin role can open the admin dashboard and add events; the dashboard shows booked seats against capacity for each event.
- A `/health` endpoint reports `UP` with the database `CONNECTED`, or `DOWN` with HTTP 503 when the database cannot be reached.

2.3 Non-Functional Requirements

- Reliability: the container has a health check, and the pipeline ends with a smoke test of the running service.
- Portability: the whole system starts with one `docker compose up -d --build` on any machine with Docker.
- Maintainability: the pipeline, the Dockerfile, the Compose file and the Nagios configuration are stored in the repository and reviewed like application code.
- Security: passwords are hashed, admin pages are protected by a role check, the container runs as an unprivileged user, and the secret key is supplied through an environment variable (`.env.example` is the only template kept in Git).

- Observability: availability of the portal, the health endpoint and the events page is checked continuously by Nagios.

2.4 Agile Setup with Jira

The project was organised with Scrum in Jira. A Jira space with the key EE was created, the team members were added as assignees, and the work was organised under one epic, EventEase Core Booking System, with stories that carry story points. Each work item has an assignee, a priority and a status that follows the board columns To Do, In Progress and Done. Commit messages begin with the Jira key of the item they belong to (for example EE-16 Redesign UI with college events), so a commit in GitHub can be matched with the work item it delivers. The GitHub repository is linked to the Jira space (Figure 2.7), so commits, branches and pull requests that carry a Jira key appear next to the work item. The link provides traceability only: the cards were moved between the board columns by the team, and no automation rule changes a status after a commit or a merged pull request.

2.5 Product Backlog

Table 2.2 lists the work items of the space as they appear in the List view (Figure 2.6): one epic and seven stories with 33 story points in total.

Key	Type	Summary	Pts	Assignee	Status
EE-14	Story	Create student registration and login	11	Miheeka Kewalramani	Done
EE-15	Epic	EventEase Core Booking System	---	Suryansh Kalchuri	Done
EE-16	Story	Browse College Events	3	Suryansh Kalchuri	Done
EE-17	Story	Book Event Tickets	3	Krish Kanal	Done
EE-18	Story	View My Bookings	3	Krish Kanal	Done
EE-19	Story	Admin Event Management	5	Krish Kanal	Done
EE-20	Story	Test EventEase Features	3	Suryansh Kalchuri	Done
EE-21	Story	Set Up GitHub Actions CI/CD	5	Miheeka Kewalramani	Done

Table 2.2: Product backlog of the Jira space

2.6 Sprint Plan

Sprint	Goal	Stories (points)
Sprint 1 (11 Oct – 25 Oct)	Implement the core student features of EventEase, including registration, login, event browsing, ticket booking and viewing bookings.	EE-14 (11), EE-16 (3), EE-17 (3), EE-18 (3) = 20 points
Sprint 2	Test the application and automate its delivery with GitHub Actions CI/CD.	EE-20 (3), EE-21 (5) = 8 points

Table 2.3: Sprint goal and scope

2.7 Definition of Done

A story is moved to Done only when all of the following hold:

1. The work was done on its own Git branch and committed with the Jira key in the message.
2. The branch reached `main` through a pull request, and the GitHub Actions check on the pull request was green.
3. The Jenkins pipeline on `main` finished with SUCCESS, including unit tests, Docker build, deployment and smoke test.
4. The behaviour was verified in the running application, and `/health` reported UP with the database CONNECTED.
5. The card was moved to the Done column of the Jira board.

2.8 The Sprint in Jira

The following figures were captured from the team's Jira space and show the sprint from its board to the completed-sprint report.

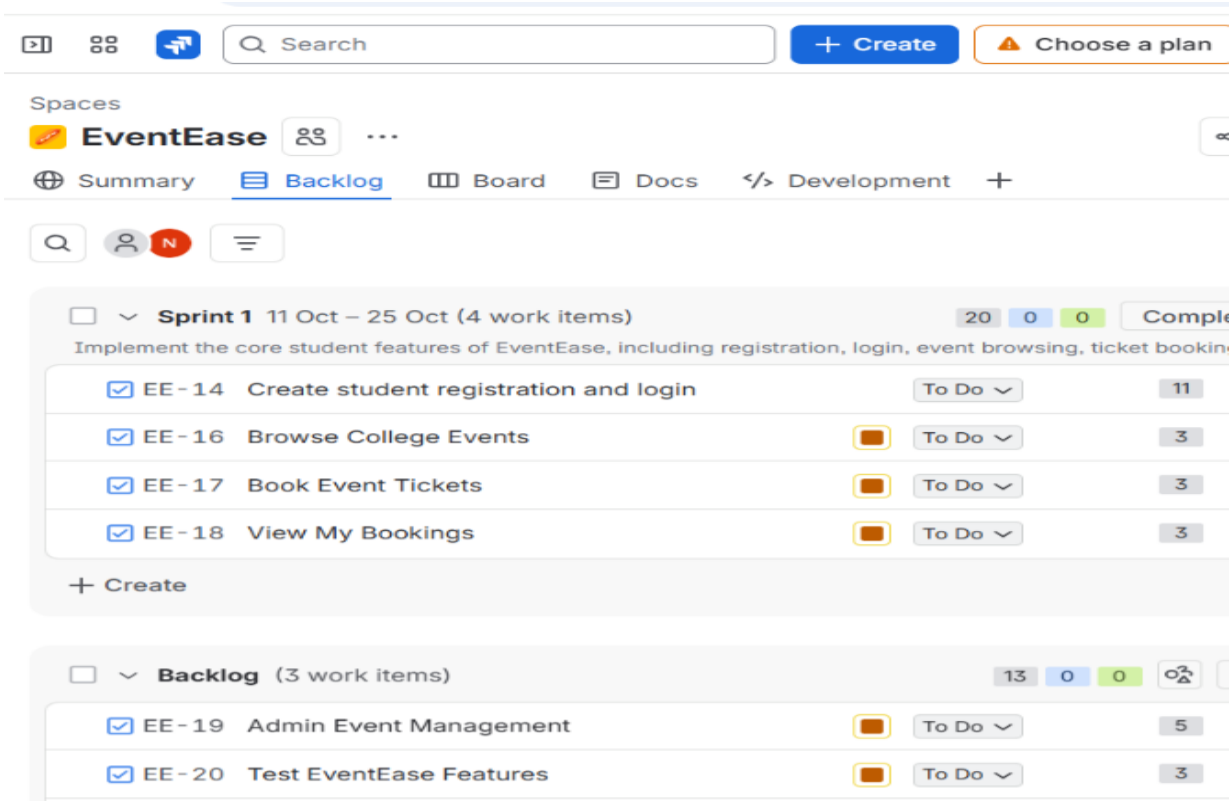


Figure 2.1: Jira backlog with Sprint 1 and the remaining backlog items

The backlog shows Sprint 1 (11 Oct – 25 Oct) with four work items worth 20 story points: EE-14 Create student registration and login (11), EE-16 Browse College Events (3), EE-17 Book Event Tickets (3) and EE-18 View My Bookings (3). Below the sprint, the backlog holds three more stories worth 13 points: EE-19 Admin Event Management, EE-20 Test EventEase Features and EE-21 Set Up GitHub Actions CI/CD.

2.8.1 Board during the sprint

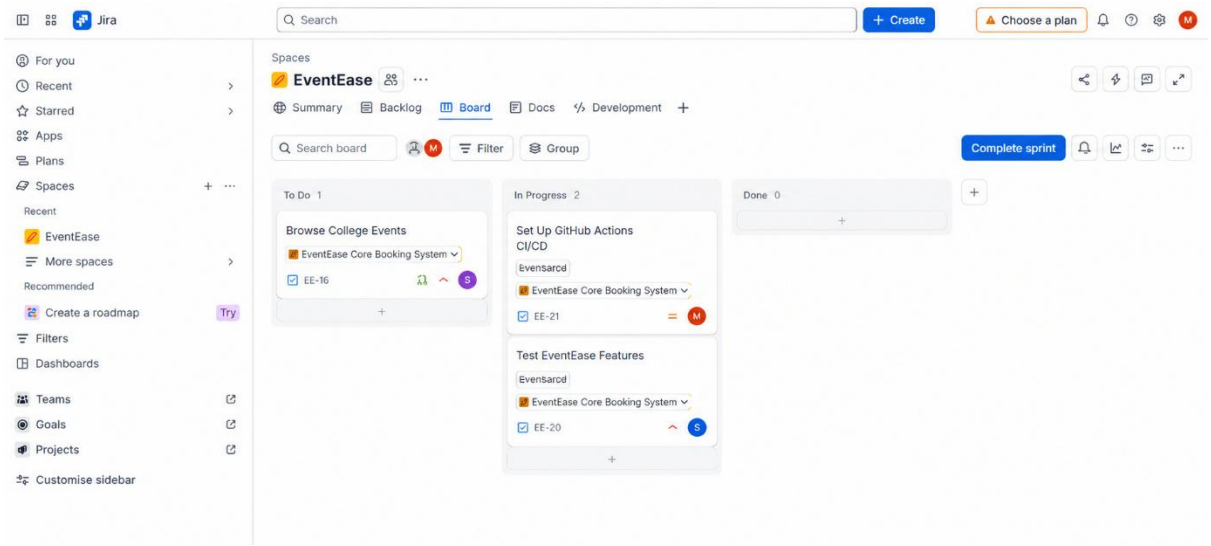


Figure 2.2: Jira board with cards moving between To Do, In Progress and Done

Figure 2.2 shows the board of Sprint 2 while the work was in progress. Three cards belong to the epic EventEase Core Booking System: Browse College Events (EE-16, Suryansh) is in To Do, and Set Up GitHub Actions CI/CD (EE-21, Miheeka) and Test EventEase Features (EE-20, Suryansh) are In Progress. Nothing is in Done yet.

2.8.2 Board near the end of the sprint

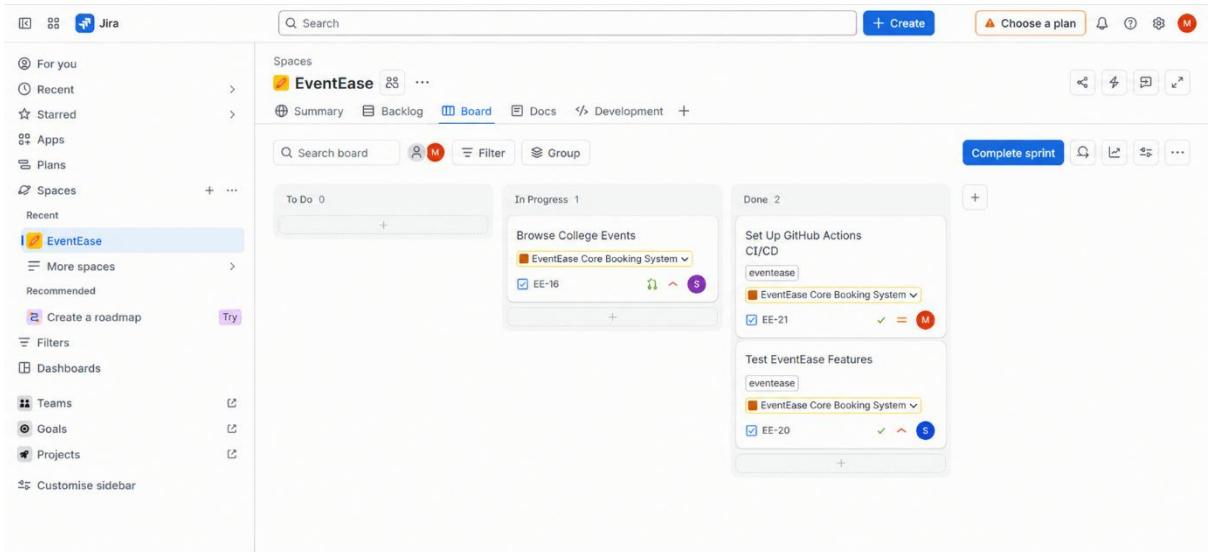


Figure 2.3: Jira board with most cards in Done

Near the end of the sprint the To Do column is empty. Set Up GitHub Actions CI/CD (EE-21) and Test EventEase Features (EE-20) have moved to Done, and only Browse College Events (EE-16) is still In Progress.

2.8.3 Completing the sprint

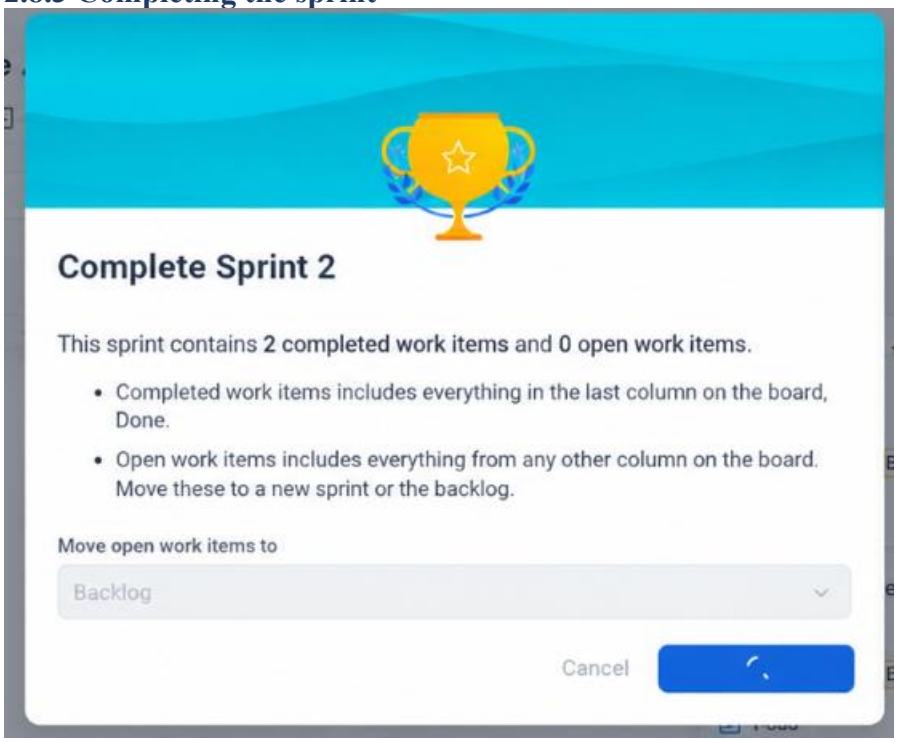


Figure 2.4: "Complete sprint" confirmation dialog

This dialog is shown when the sprint is closed with the Complete sprint button. It states how many work items the sprint contains and that all of them are completed, so nothing has to be moved back to the backlog. Sprint 2 was closed with 2 completed work items and 0 open work items.

2.8.4 Summary view

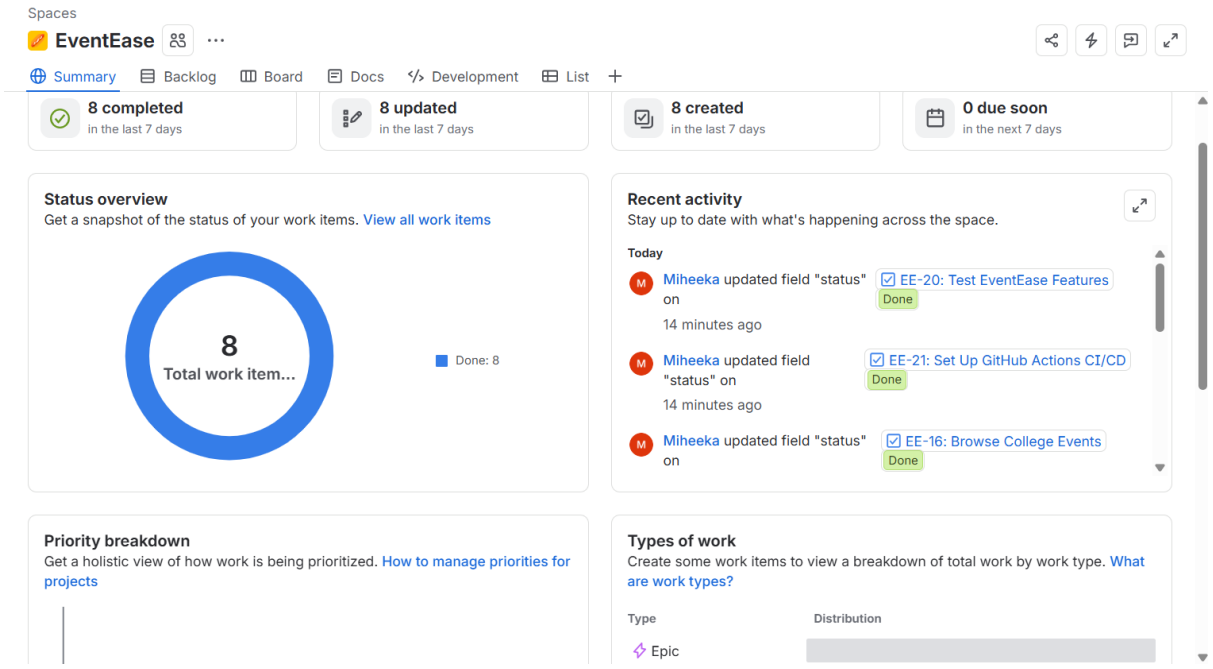


Figure 2.5: Jira Summary view with status overview and recent activity

2.8.5 List view

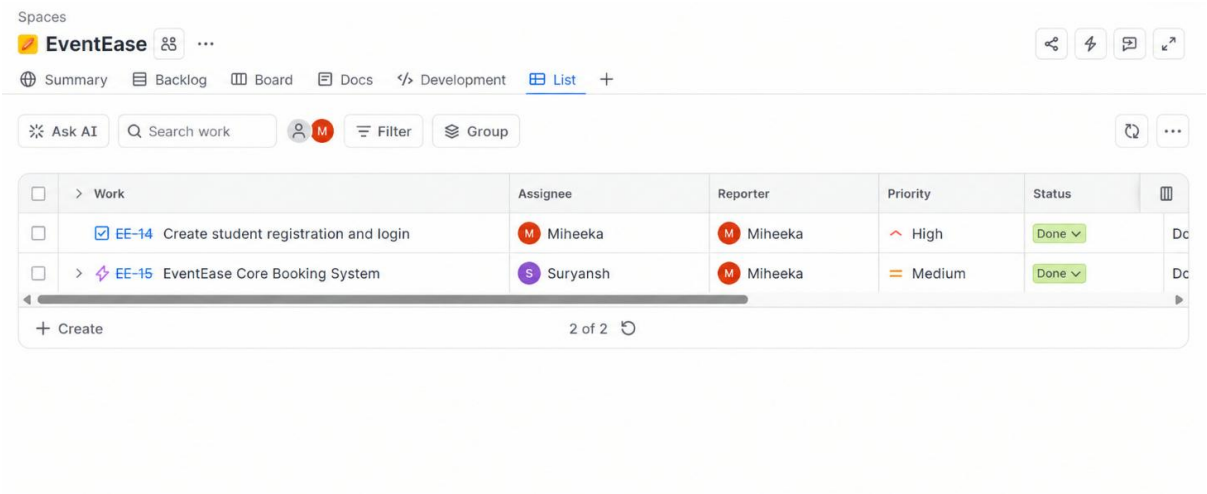


Figure 2.6: Jira List view with assignees, priority and status

The List view puts every work item in one table with its assignee, reporter, priority and status. It is the final evidence that every story has an owner and is Done. No epic stays in To Do: the epic EE-15 EventEase Core Booking System is Done and is assigned to Suryansh. EE-14 is listed on its own, and the other six stories sit under the epic and appear when it is expanded.

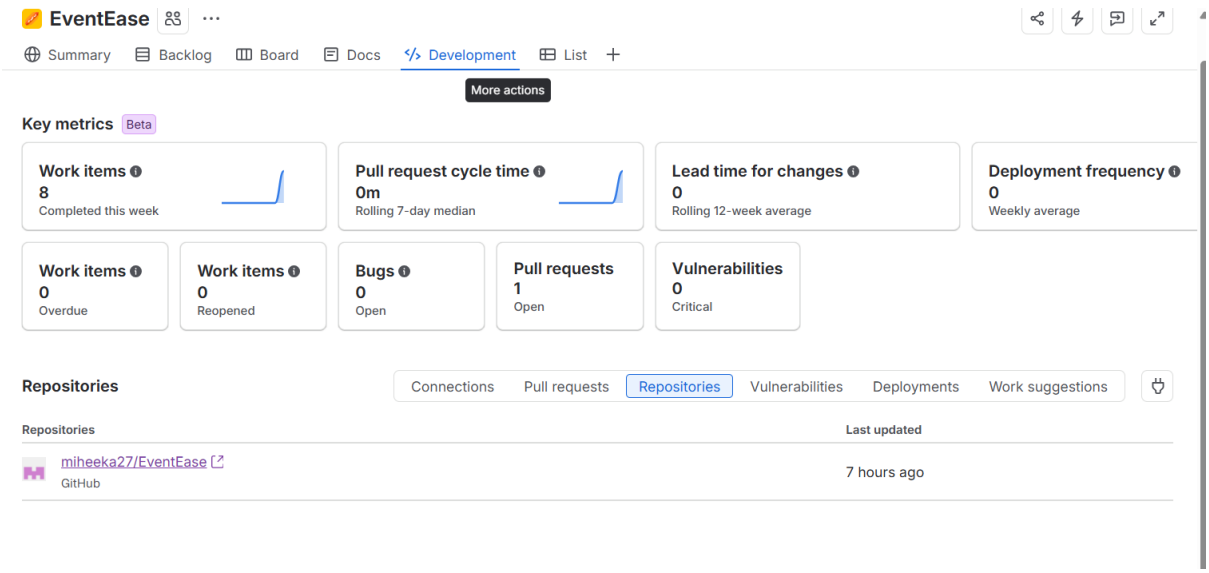


Figure 2.7: Jira Development page showing the linked GitHub repository miheeka27/EventEase

The Development tab of the Jira space shows that the GitHub repository miheeka27/EventEase is connected to the project, so activity in the repository appears next to the work items it belongs to. Because every commit message begins with a Jira key such as EE-16 or EE-17, commits, branches and pull requests can be matched with the story they deliver. The page also reports the 8 work items completed this week. The link provides traceability only: the cards were moved between To Do, In Progress and Done by the team, and no automation rule changes a status after a commit or a merged pull request.

3. System Design

3.1 Architecture

EventEase is a single Flask application packaged as one container (Figure 3.1). The browser talks to Gunicorn on port 5000, which serves the HTML pages, handles registration, login and booking, and answers the `/health` probe. Data is kept in an SQLite file, stored in the Docker volume `eventease-data` mounted at `/data`, so bookings survive when the container is restarted or rebuilt.

Around the application sit the delivery and monitoring tools. The source code is kept in the GitHub repository miheeka27/EventEase, where a GitHub Actions workflow runs the tests. Jenkins, running on the development machine on port 8080, checks out `main`, runs the tests, builds the image, deploys it with Docker Compose and smoke-tests the result. Nagios Core, started from a second Compose file and available on port 8081, watches the running application from outside.

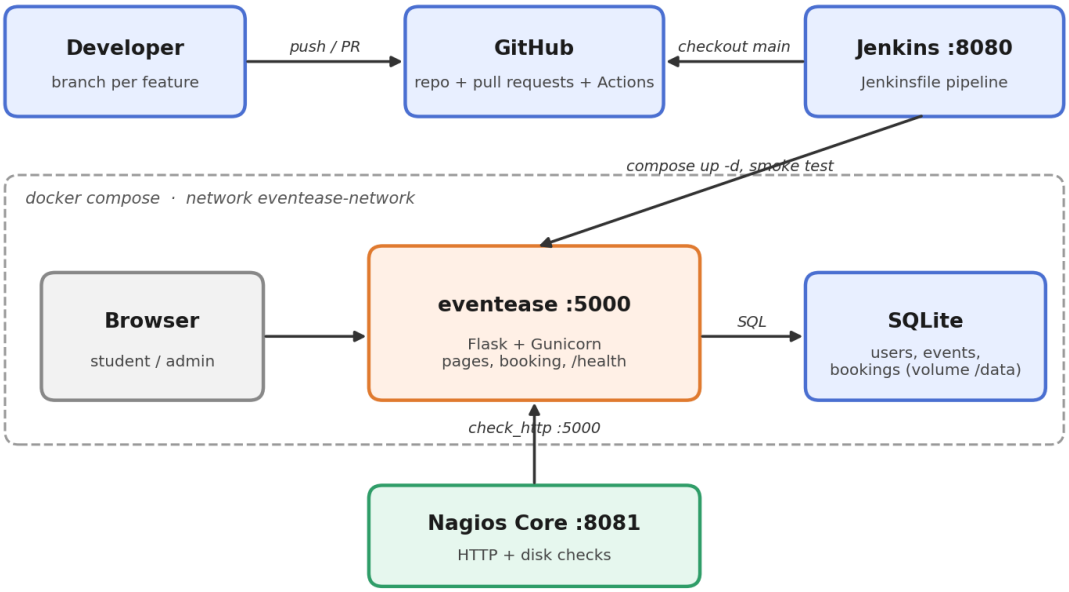


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	Jinja2 templates, CSS (Poppins font), a little JavaScript	Server-rendered pages need no build step; the events page filters instantly in the browser
Backend	Python, Flask 3.1, Gunicorn 23	Small framework with routing, sessions and templates built in
Database	SQLite (`database.db`, a Docker volume in production)	No separate database server to run; enough for one college
Testing	pytest with Flask's test client	Tests need no running server and use a temporary database

Layer	Technology	Reason for choice
Version control	Git and GitHub, GitHub Actions	Branches, pull requests and an automatic test run on every push
CI/CD	Jenkins (Jenkinsfile in the repository)	Pipeline as code: checkout, test, build, deploy, smoke test
Containers	Docker and Docker Compose	Identical environment on every machine; one command to start
Monitoring	Nagios Core	State-based checks of the service and the host
Project tracking	Jira	Epics, stories, sprint board, sprint report

Table 3.1: Technology stack

3.3 Database Design

Three tables carry the data (Figure 3.2). A user has a name, a unique e-mail address, a hashed password and a role, which is `student` by default or `admin`. An event has a name, date, venue, price, capacity, description, organiser (club), category and the name of its poster image. A booking links one user to one event and records the number of tickets. One user can make many bookings and one event can receive many bookings; the seats left for an event are its capacity minus the sum of the tickets in its bookings.

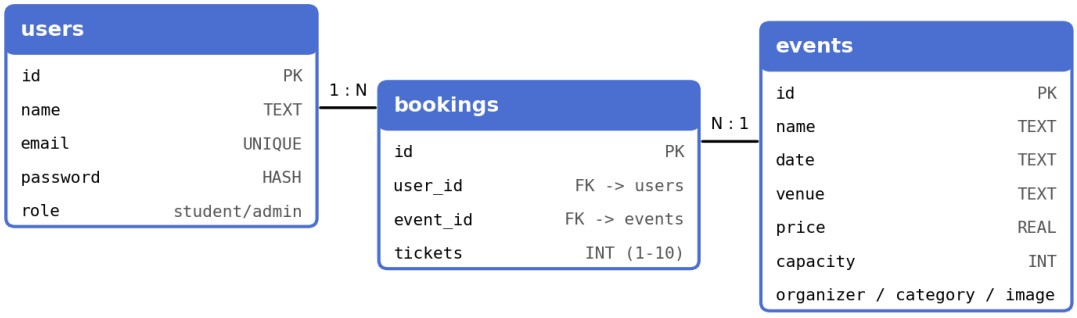


Figure 3.2: Entity-relationship diagram

3.4 Routes

Route	Method	Purpose	Access
/	GET	Home page with statistics and all event cards	Public
/events	GET	Event list with search and category filters	Public
/event/<id>	GET	Event details, seats left, booking box (404 if missing)	Public
/register	GET, POST	Create an account (validates e-mail and password length)	Public
/login, /logout	GET, POST	Start or end a session	Public

Route	Method	Purpose	Access
/book/<id>	POST	Book 1 to 10 tickets if seats are left	Logged-in student
/my-bookings	GET	List the student's bookings with totals	Logged-in student
/admin	GET	Dashboard with booked seats per event	Admin only
/admin/events/add	GET, POST	Add a new event	Admin only
/health	GET	`UP` and `CONNECTED`, or `DOWN` with HTTP 503	Public (used by Docker, Jenkins, Nagios)

Table 3.2: Routes of the application

3.5 Booking Rules

The most important business logic is booking. The server, not the browser, enforces the rules, so they hold for any client.

Situation	Result
User is not logged in	Redirected to the login page with a message
Ticket count is not a number, or is below 1 or above 10	Rejected with a message; nothing is saved
Event does not exist	HTTP 404
Tickets already booked plus the new tickets exceed the capacity	Rejected with "not enough seats available"
Otherwise	Booking is saved and the student is taken to My Bookings

Table 3.3: Booking rules enforced by the application

3.6 Security and Configuration Design

- Passwords are stored as hashes, never as plain text.
- Admin pages check the role stored in the session, and a student who opens `/admin` is redirected to the login page.
- The container runs as an unprivileged user (`appuser`, id 1001), not as root.
- The secret key comes from the environment variable `SECRET_KEY`; only the template `.env.example` is kept in Git and `.gitignore` keeps `.env` and `database.db` out of the repository.
- Only port 5000 is published to the host; the container joins the private bridge network `eventease-network`.

4. Application Implementation

This chapter walks through the redesigned application using the team's own screenshots. The interface was redesigned with a card layout, a hero section and a responsive navigation bar, and it is loaded with nine college events.

4.1 Home Page

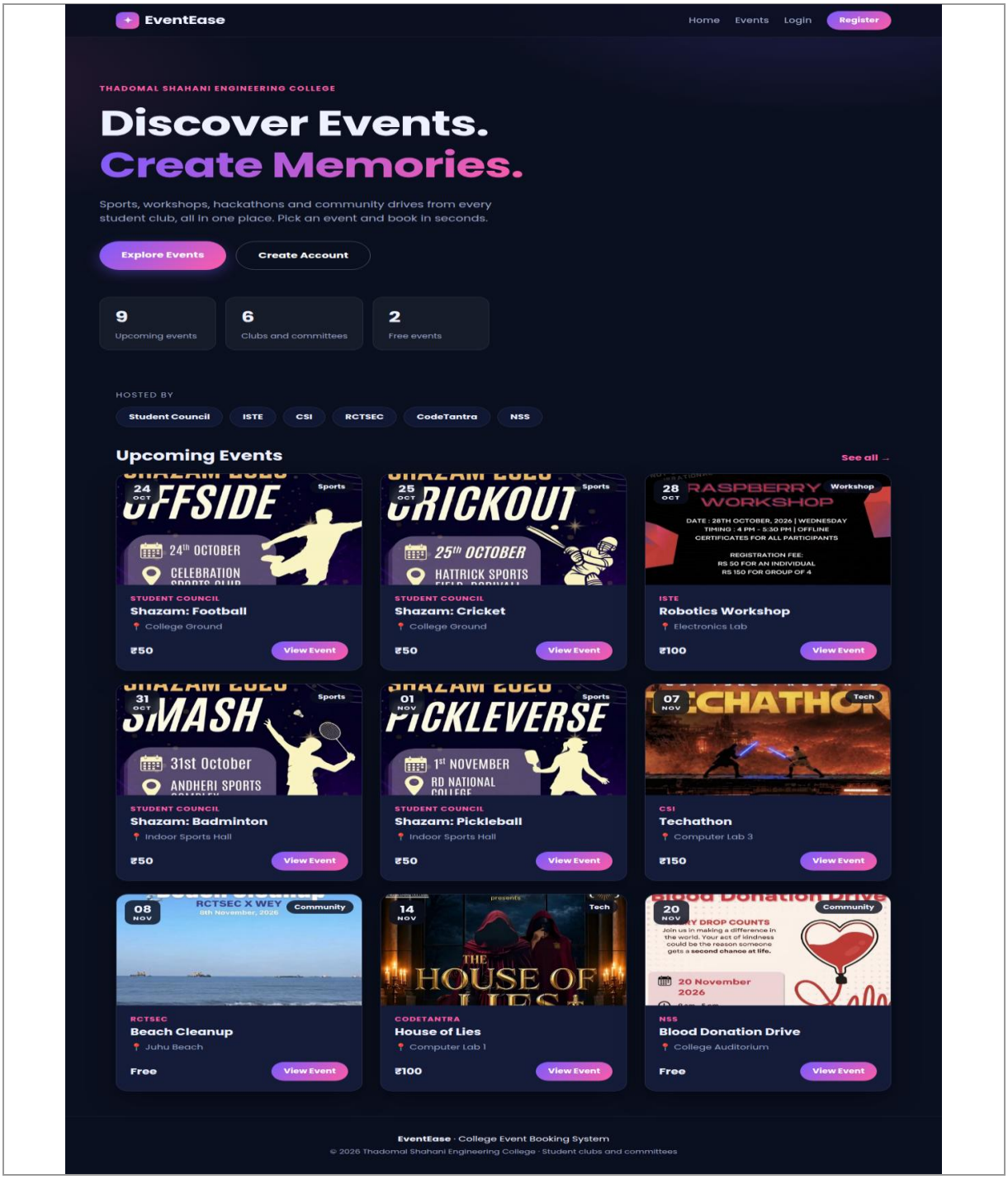


Figure 4.1: Home page of EventEase

The home page opens with a hero section that introduces the platform, followed by three figures calculated from the data: the number of upcoming events (9), the number of clubs and

committees (6) and the number of free events (2). A row of pills names the hosting clubs, and below it every event is shown as a card with its poster, date, venue and price.

4.2 Events Page with Search and Filters

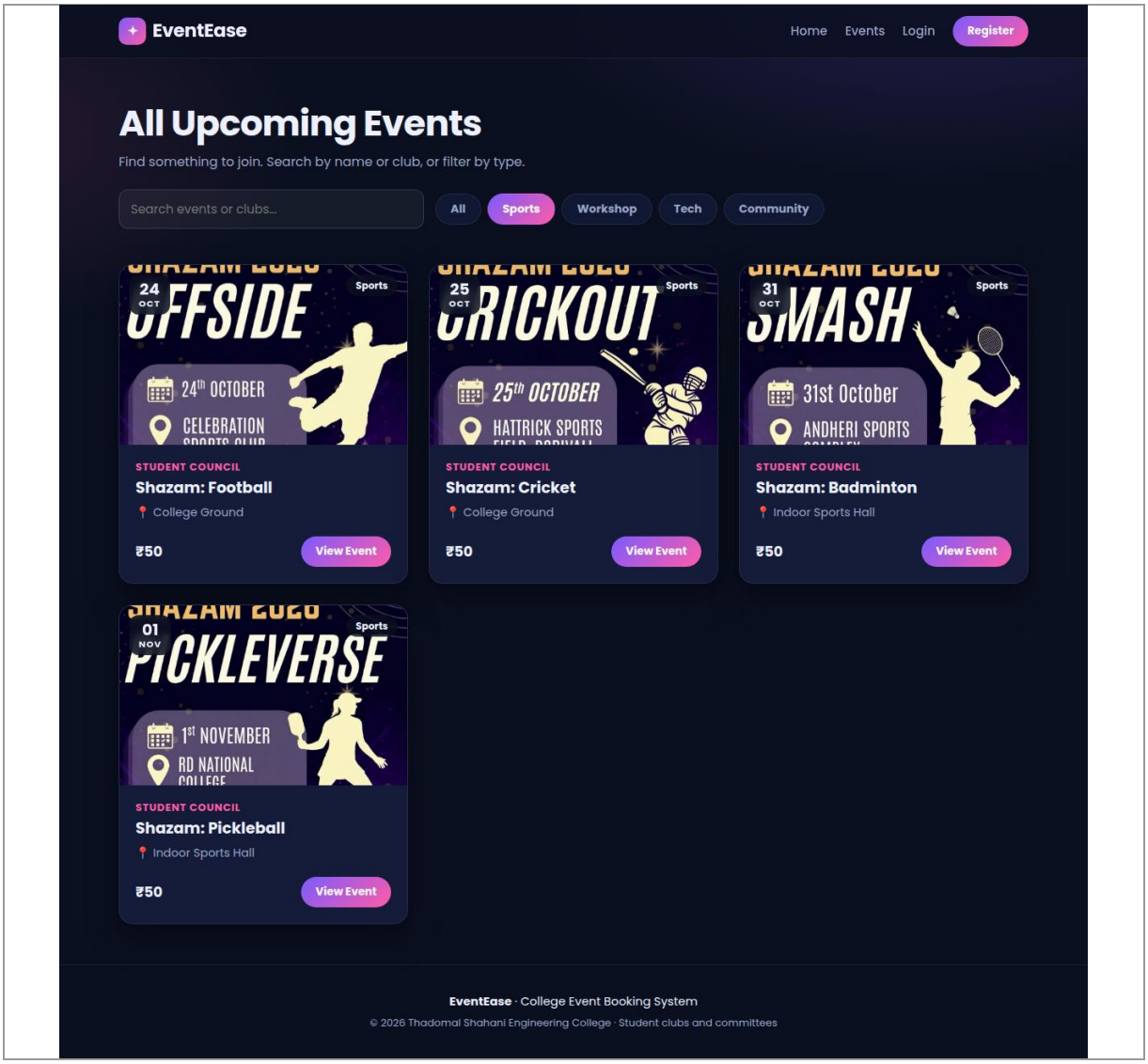


Figure 4.2: Events page filtered by the Sports category

The events page lists all events. The search box narrows the list as the student types, and the filter buttons All, Sports, Workshop, Tech and Community show only the events of one category. The filtering is done in the browser with JavaScript, so the page does not reload. When nothing matches, a short message is shown instead of an empty page.

4.3 Event Page and Booking Box

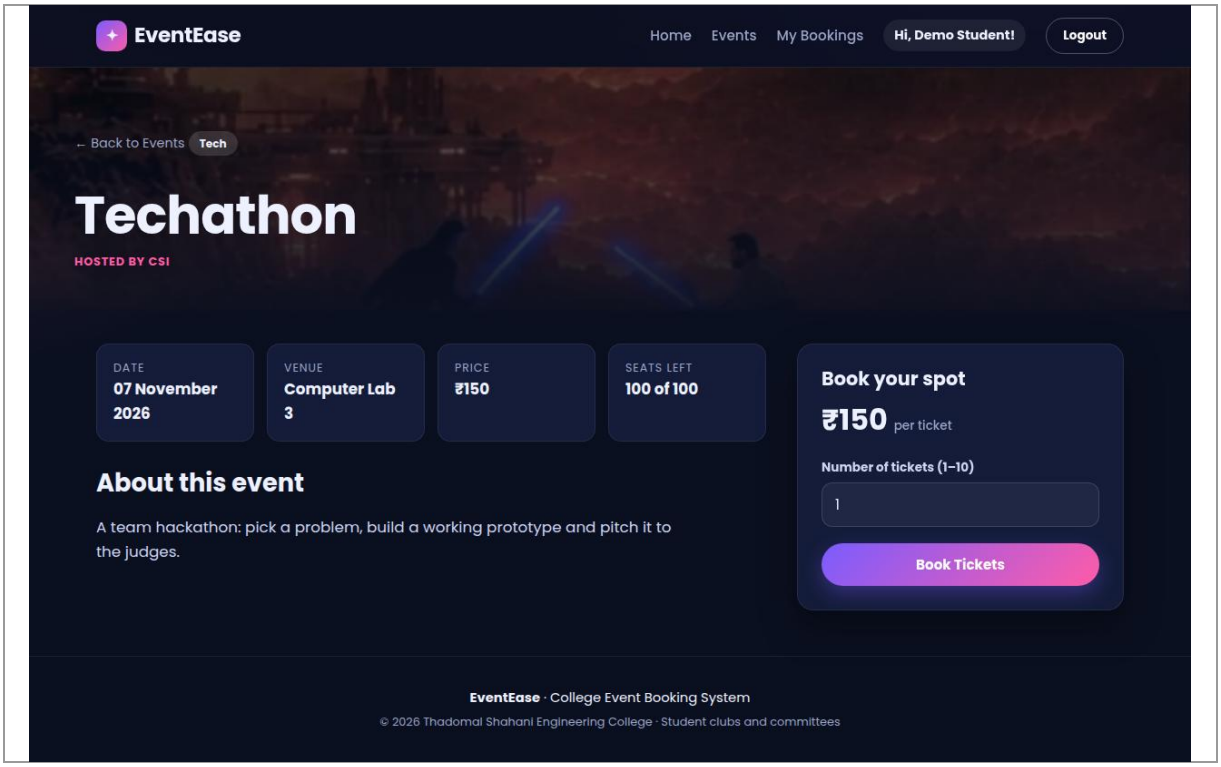


Figure 4.3: Event page of Techathon with the "Book your spot" box

An event page shows the date, venue, price and the seats left out of the capacity, followed by the description. The booking box on the right asks for the number of tickets (1 to 10). A visitor who is not logged in sees a Login to Book button instead, and a sold-out event shows a Sold out message.

4.3.1 Registration and Login

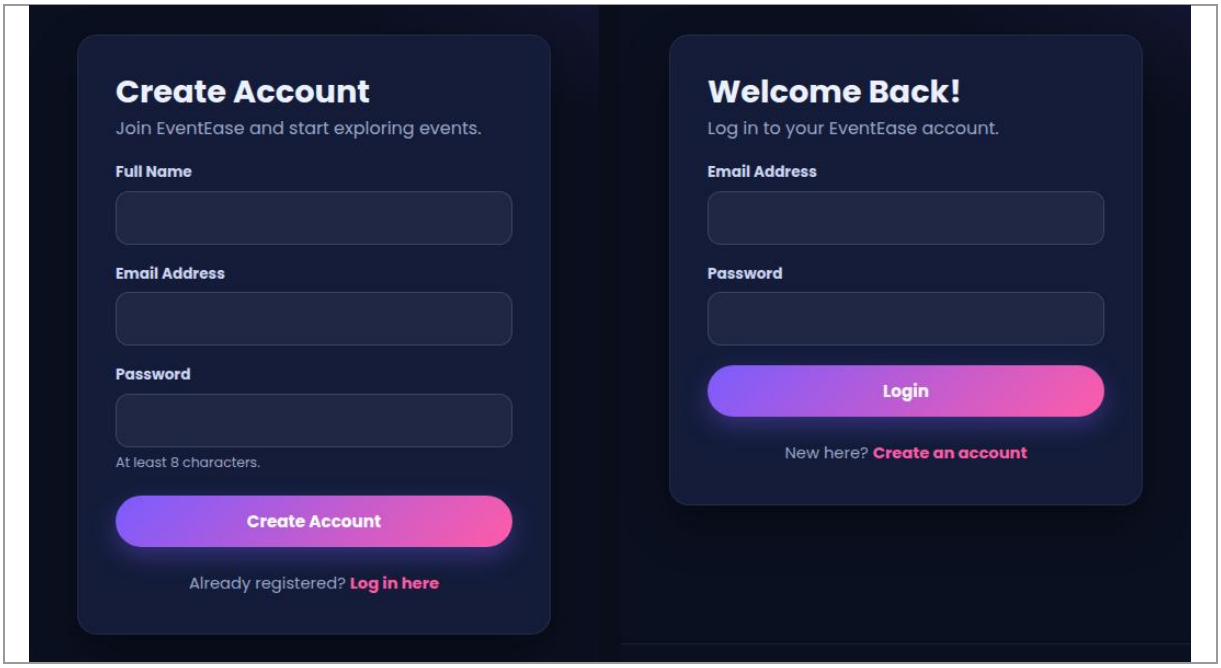


Figure 4.4: Register page (left) and Login page (right)

The register form validates the e-mail address and requires a password of at least eight characters. The password is stored as a hash, and registering the same e-mail twice is rejected. After logging in, the navigation bar shows the student's name, My Bookings and Logout.

4.4 My Bookings

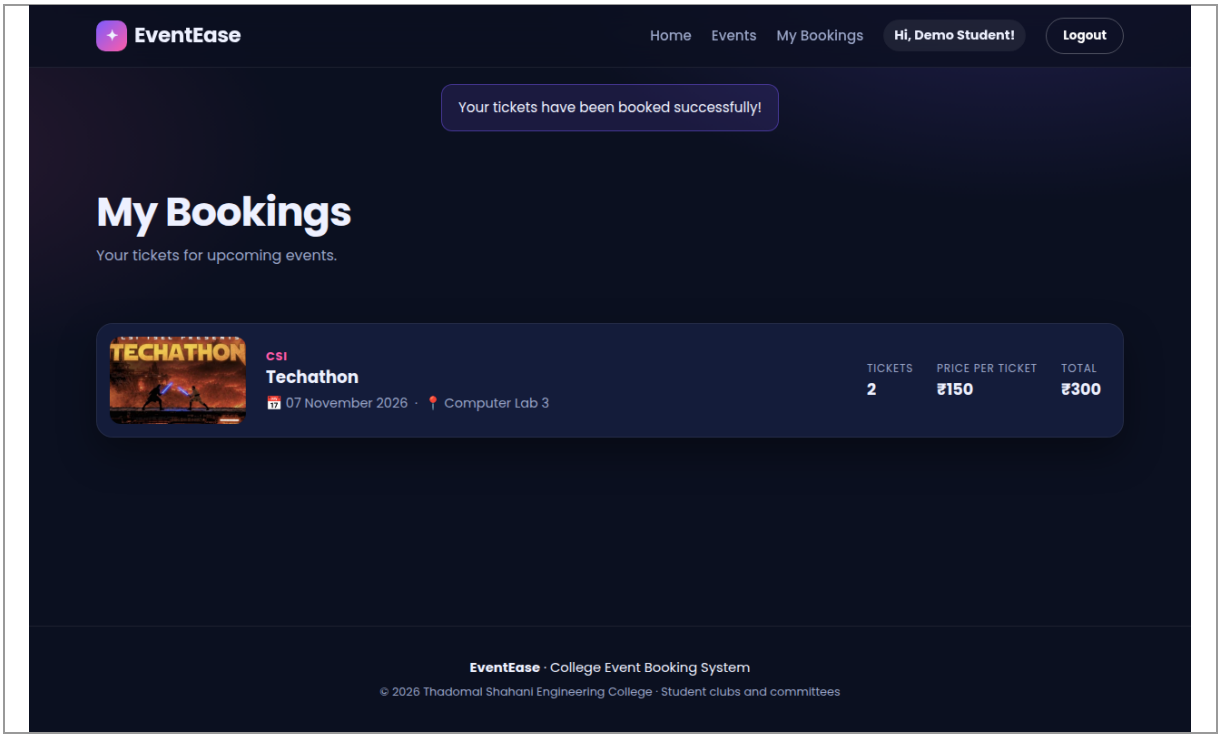


Figure 4.5: My Bookings page after a successful booking

After a booking the student is taken to My Bookings, which lists each booking with the poster, club, date, venue, number of tickets, price per ticket and the total. The booking shown is Techathon (CSI) on 07 November 2026: 2 tickets at ₹150 each, a total of ₹300, confirmed by the message "Your tickets have been booked successfully!".

4.5 Admin Dashboard

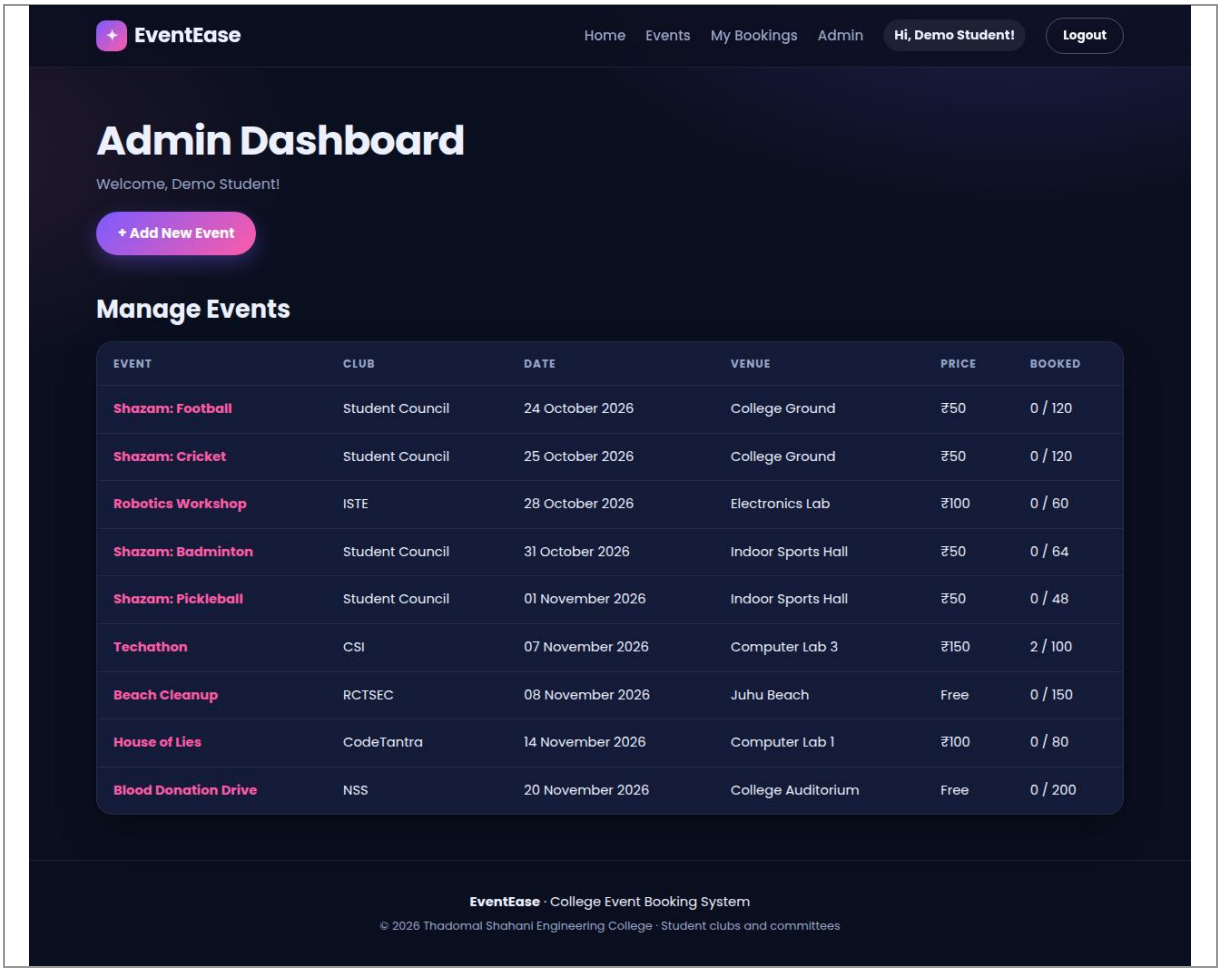


Figure 4.6: Admin dashboard with booked seats for each event

The admin dashboard is available only to a user with the admin role, which is granted with `python create_admin.py`. It lists every event with its club, date, venue, price and the seats booked against the capacity (for example 2 / 100), and a button leads to the Add Event form. The form takes the name, club, category, date, venue, price, capacity and description, and validates that the capacity is at least 1 and the price is not negative.

4.6 Application Logic

The application is a single file, `app.py`, that registers the routes of Table 3.2 and uses `database.py` for the connection and for creating the tables. On start-up the database is created and, if the events table is empty, the nine college events are inserted. An older database is upgraded automatically by adding the newer columns, and the original placeholder events are removed unless somebody has booked them. The poster of each event is found by name in `static/images`, where a real photograph with the event's name overrides the drawn poster. The health route runs `SELECT 1` against the database and returns `{"status": "UP", "database": "CONNECTED"}`, which the Docker health check, the Jenkins smoke test and Nagios all rely on.

5. DevOps Implementation

This chapter describes how each lab experiment was applied to EventEase. Figure 5.1 summarises the delivery pipeline that ties them together: work starts as a Jira story, is committed to a branch in Git, reaches `main` through a pull request, is tested by GitHub Actions and Jenkins, built and deployed as a Docker container, and is then watched by Nagios. The Jira part was described in Chapter 2.

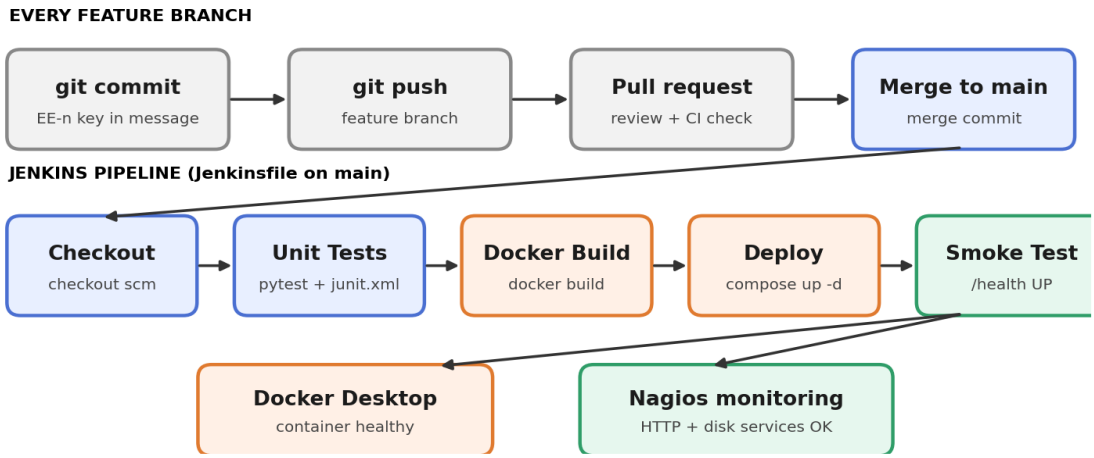


Figure 5.1: Delivery pipeline from commit to monitoring

5.1 Branching Strategy

The redesign of the website and the DevOps files were deliberately kept on separate branches, so each could be reviewed and merged on its own. Every commit message begins with the Jira key of its story.

Branch	Content	Jira key	Merged by
feature/EE-16-ui-redesign	Redesigned templates, stylesheet, college events and posters	EE-16	Pull request #2
feature/EE-17-devops-files	Dockerfile, Compose files, Jenkinsfile, Nagios configuration, health route and tests	EE-17	Pull request #3

Table 5.1: Feature branches and their pull requests

5.2 Version Control with Git and GitHub

All source code, the Dockerfile, the Compose files, the Jenkinsfile, the GitHub Actions workflow and the Nagios configuration are kept in one GitHub repository. A branch reaches `main` only through a pull request.

5.2.1 Branches and commit history

```
(venv) C:\Users\Admin\Desktop\EE-MPR>git branch -a
  feature/EE-16-ui-redesign
  feature/EE-17-devops-files
* main
  remotes/origin/EE-14-student-registration
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/EE-16-ui-redesign
  remotes/origin/feature/EE-17-devops-files
  remotes/origin/main
```

Figure 5.2: Output of git branch -a

```
(venv) C:\Users\Admin\Desktop\EE-MPR>git log --graph --pretty=format:"%h - %d %s [%an]" -n 12
*   5df4636 -  (HEAD -> main, origin/main, origin/HEAD) Merge pull request #3 from miheeka27/feature/EE-17-devops-files [miheeka27]
|\
| * 89ed4dc -  (origin/feature/EE-17-devops-files, feature/EE-17-devops-files) EE-17 Add Docker, Jenkins and Nagios files [miheeka27]
|/
*   9d20331 -  Merge pull request #2 from miheeka27/feature/EE-16-ui-redesign [miheeka27]
|\
| * 2298687 -  (origin/feature/EE-16-ui-redesign, feature/EE-16-ui-redesign) EE-16 Redesign UI with college events [miheeka27]
|/
* ccbc310 -  Initial commit: EventEase college event booking system [Miheeka]
```

Figure 5.3: Commit graph showing the feature branches merged into main

`git branch -a` lists the local branches and the remote-tracking branches under `remotes/origin`. The graph drawn by `git log --graph` shows the feature branches splitting off `main` and returning through merge commits, with the Jira key at the start of every message. The branch feature/EE-16-ui-redesign carries the commit 2298687 (EE-16 Redesign UI with college events) and was joined to main by the merge commit 9d20331, pull request #2. The branch feature/EE-17-devops-files carries the commit 89ed4dc (EE-17 Add Docker, Jenkins and Nagios files) and was joined by the merge commit 5df4636, pull request #3. Both branches start from the initial commit ccbc310. The remote also has the branch EE-14-student-registration.

5.2.2 The repository on GitHub

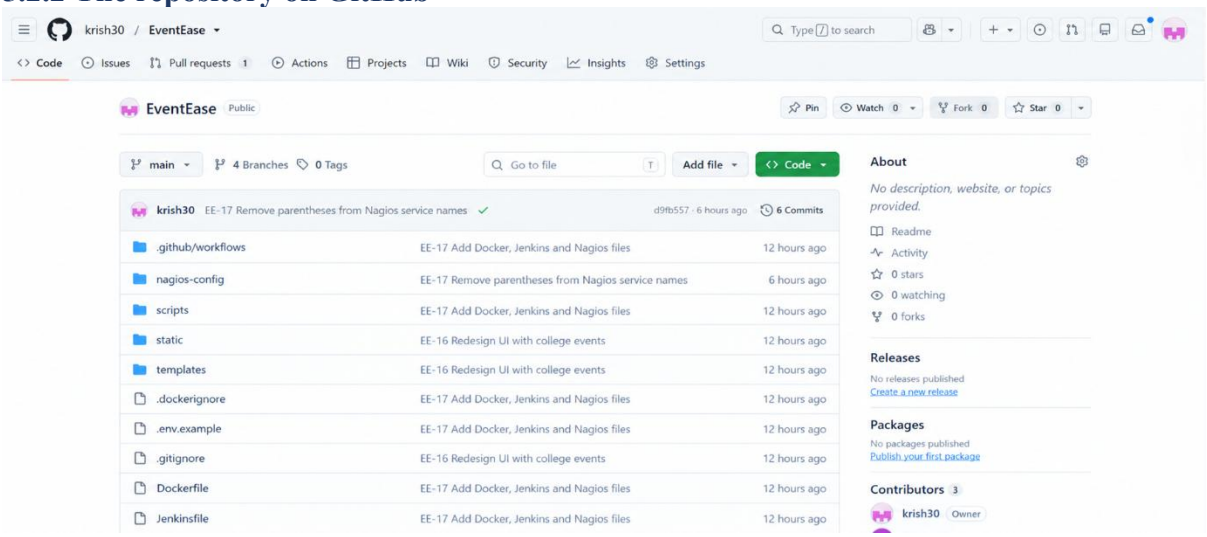


Figure 5.4: GitHub repository page of EventEase

The repository page shows the number of branches and commits and the file list. The Dockerfile, `docker-compose.yml`, `Jenkinsfile`, `nagios-config/` and `.github/workflows/` sit next to the application code, so the way the software is built and deployed is versioned together with the software itself. At the time of the screenshot the repository had 4 branches and 6 commits; the latest commit, d9fb557, renames the Nagios services (EE-17).

5.2.3 Pull requests and the GitHub Actions check

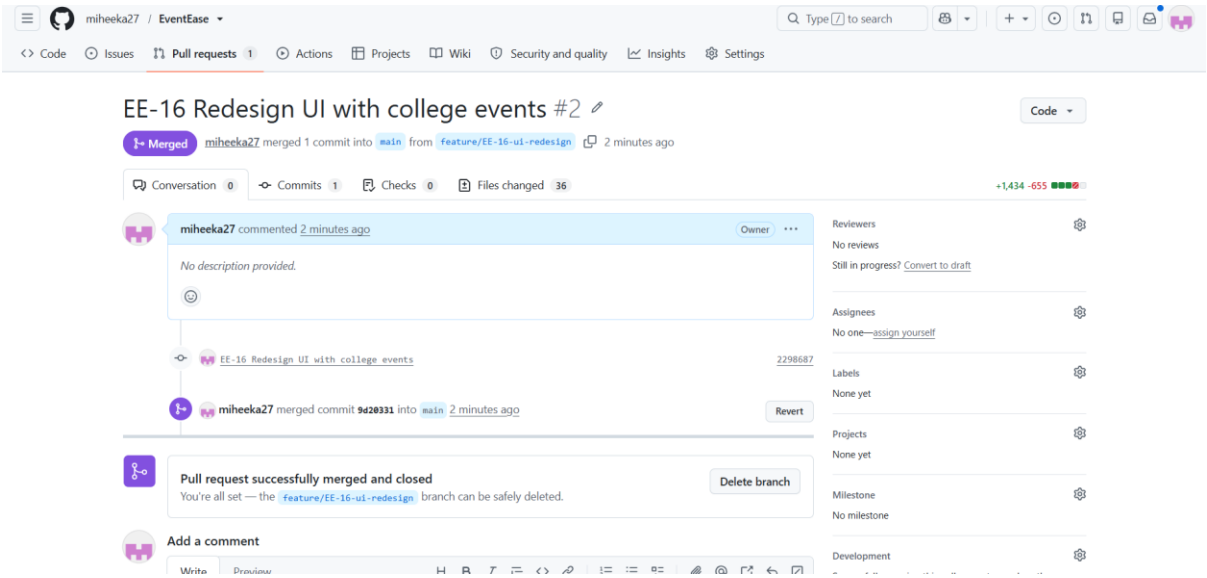


Figure 5.5: Merged pull request for the website redesign

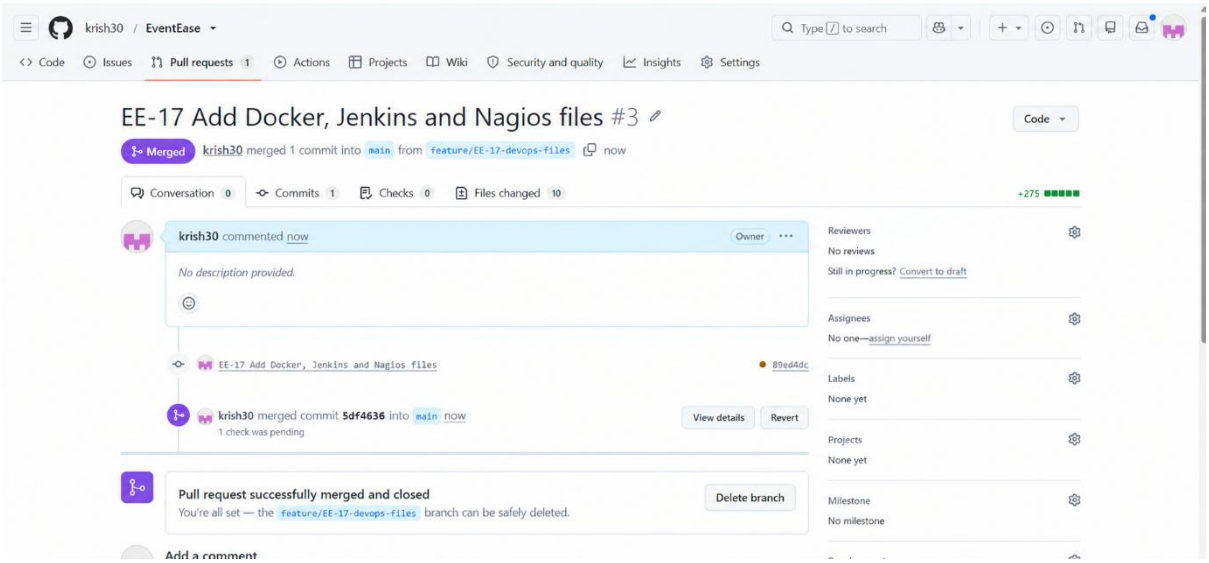


Figure 5.6: Merged pull request for the DevOps files

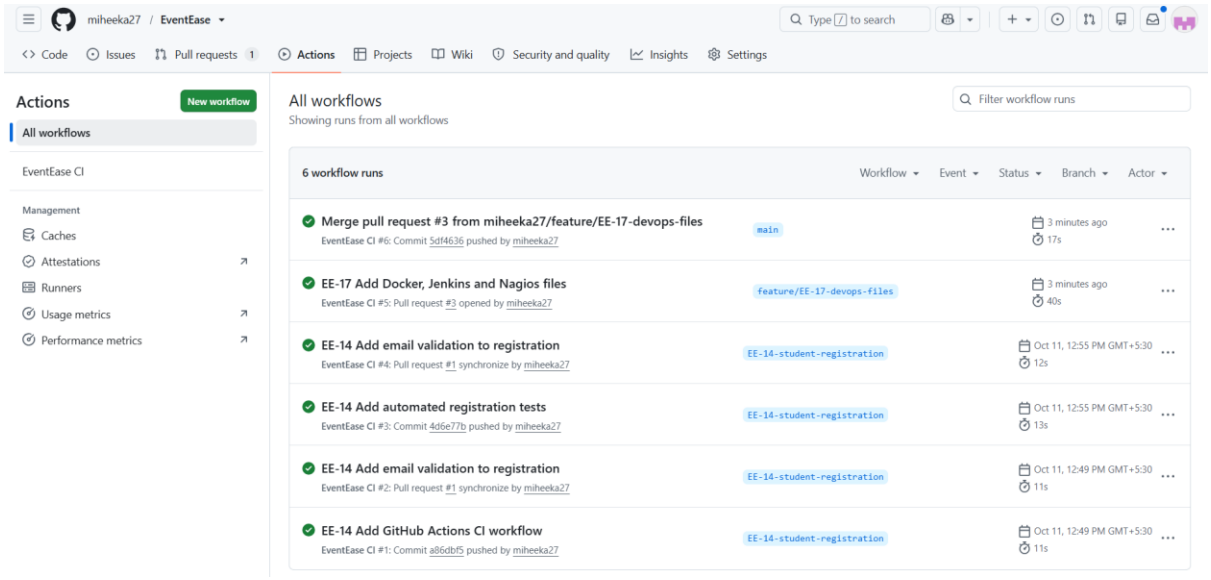


Figure 5.7: GitHub Actions run "EventEase CI" with a green check

Each branch was merged through its own pull request. The workflow `EventEase CI` runs on every push to `main` and on every pull request into `main`: it checks out the code, sets up Python 3.13, installs `requirements.txt` and runs `python -m pytest -v` on an Ubuntu runner. The green check on the pull request shows that the tests passed before the merge. A merged pull request is also the trigger for the Jenkins run, because Jenkins builds `main`.

5.3 Containerisation with Docker and Docker Compose

The application has one Dockerfile, which is built into the image `eventease:latest`.

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 DATABASE_PATH=/data/database.db
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN groupadd -g 1001 appgroup && useradd -u 1001 -g appgroup -m appuser \
    && mkdir -p /data && chown -R appuser:appgroup /app /data
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "4",
"app:app"]
```

The base image `python:3.11-slim` keeps the image small. The dependency list is copied and installed before the application code, so the slow `pip install` layer is reused from the cache whenever only the source code changes. A non-privileged user `appuser` owns the application and the data folder, so the service does not run as root. Gunicorn runs one worker with four threads, because the sample events are inserted when the application starts and several workers could insert them at the same time.

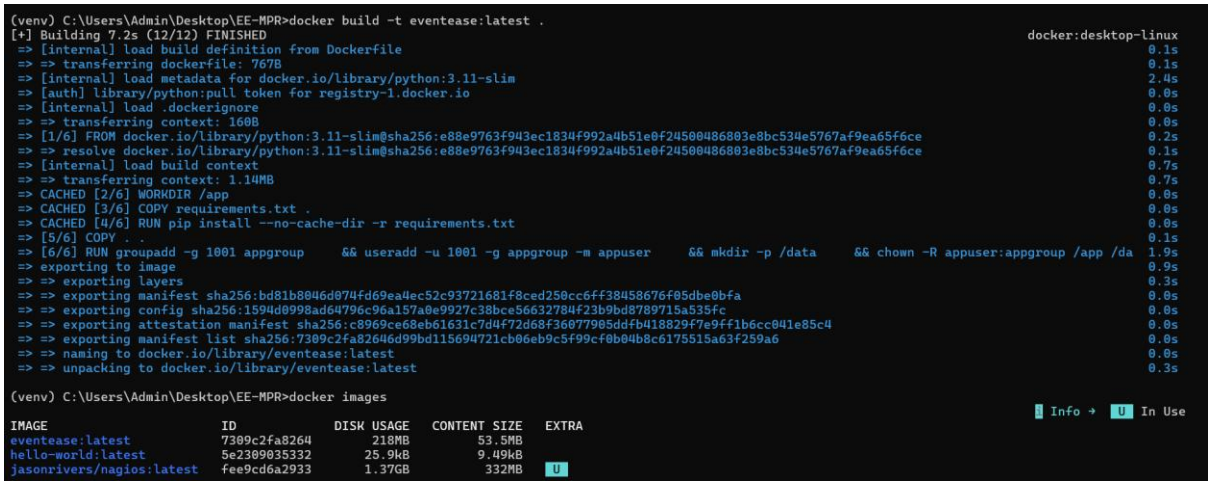


Figure 5.8: docker build and docker images

`docker build` printed one step for every instruction of the Dockerfile and finished in 7.2 seconds, with the dependency layers taken from the cache. docker images then lists eventease:latest with a disk usage of 218 MB (53.5 MB of content), next to the Nagios image jasonrivers/nagios:latest.

5.3.1 Docker Compose

The service is described once in `docker-compose.yml` and started with a single command. The file defines the service `eventease` with port mapping 5000:5000, the environment variables `DATABASE_PATH` and `SECRET_KEY`, the named volume `eventease-data` mounted at `/data`, a health check and the private network `eventease-network`.

Item	Setting	Purpose
Image / container	`eventease:latest` / `eventease`	Image built from the Dockerfile
Port	5000:5000	Web portal and `/health`
Volume	`eventease-data` at `/data`	Keeps the SQLite file when the container is rebuilt
Health check	Python request to `/health` every 15 s, 3 retries, 10 s start period	Marks the container healthy only when the application answers
Restart policy	`unless-stopped`	Container comes back after a restart of Docker
Network	`eventease-network` (bridge)	Private network of the stack

Table 5.2: Configuration of the Compose service

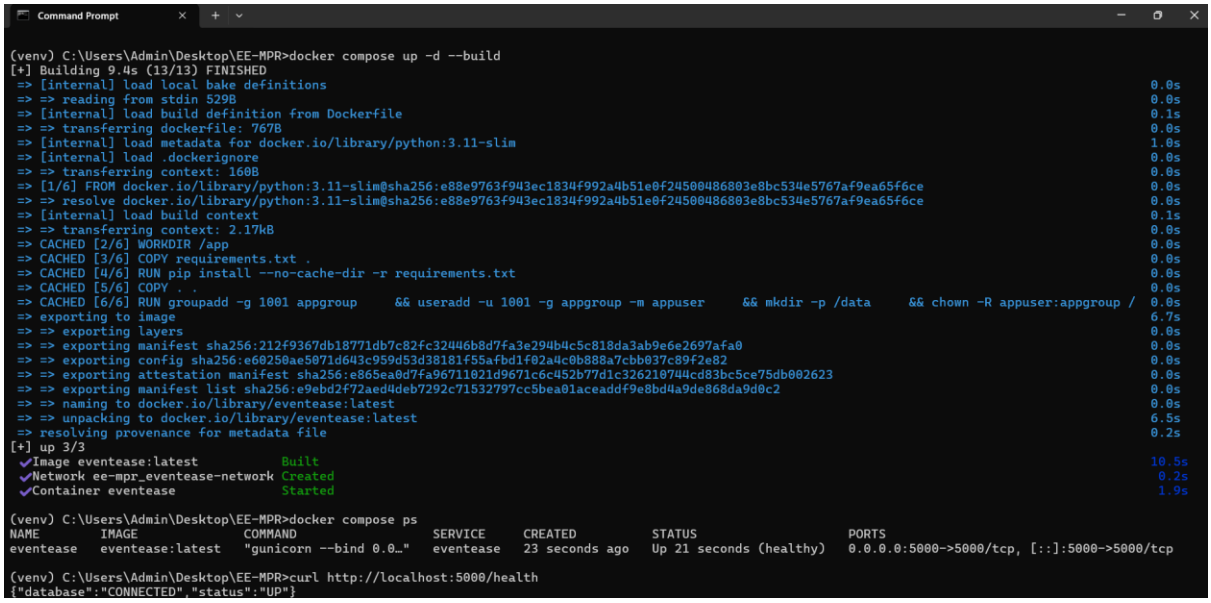


Figure 5.9: docker compose up, docker compose ps and the health request

`docker compose up -d --build` rebuilt the image, created the network and the volume and started the container `eventease` in the background. `docker compose ps` shows the container with the status "Up … (healthy)" and the port mapping `0.0.0.0:5000->5000/tcp`; "healthy" means that the health check is passing, not just that the process has started. `curl http://localhost:5000/health` returns `{"database":"CONNECTED","status":"UP"}`, which shows that the application is up and can reach its database.

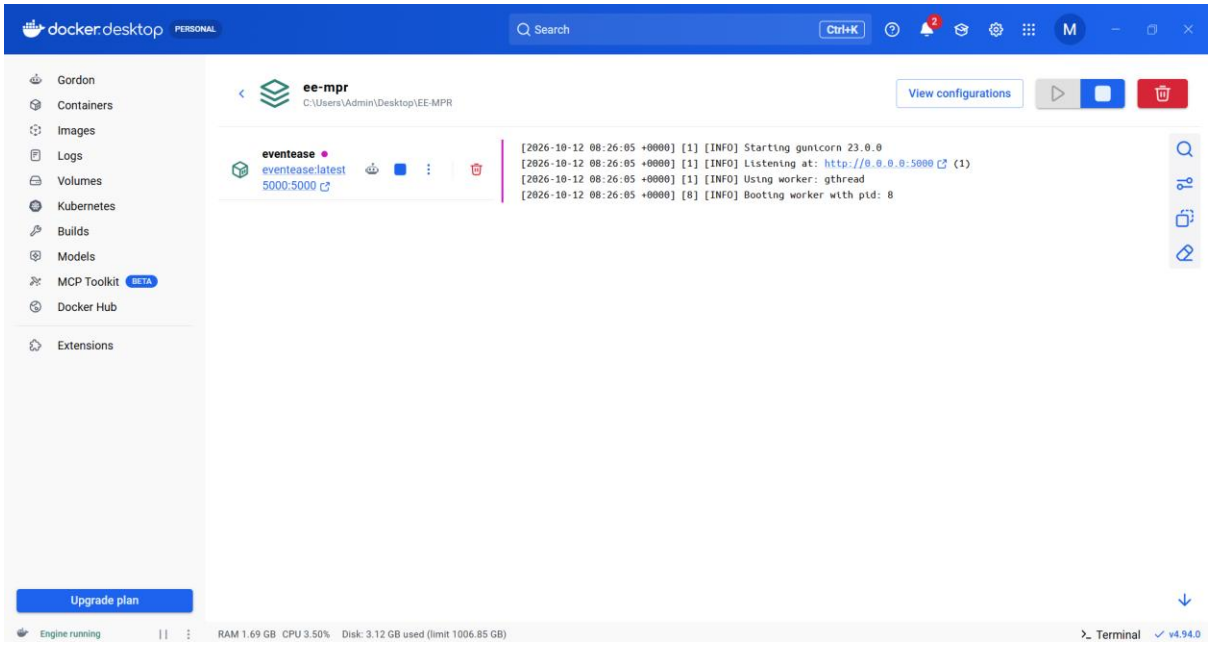


Figure 5.10: Docker Desktop showing the running eventease container

Docker Desktop shows the same container as running, with the port 5000 and its log pane, where the repeated `GET /health` requests with status 200 are the health checks that Docker runs inside the container.

5.4 Continuous Integration and Deployment with Jenkins

Jenkins automates what the team would otherwise do by hand after every change. The pipeline is declared in the `Jenkinsfile` in the root of the repository, so it is versioned and reviewed like any other code. Jenkins builds the `main` branch, so the pull requests are merged before the pipeline is run.

Stage	What it does	Purpose
Checkout	`checkout scm`: clones the repository and branch configured in the job	Build exactly the code that is on `main`
Unit Tests	Creates a virtual environment, installs `requirements.txt`, runs `python -m pytest -v --junitxml=test-reports\junit.xml`; the JUnit report is recorded	Catch defects before anything is built or deployed
Docker Build	`docker build -t eventease:latest .`	Make sure the image can be built from the repository
Deploy	`docker compose up -d --build`, then `docker compose ps`	Start the stack from the new image
Smoke Test	`scripts/smoke_test.ps1` calls `/health` up to 12 times, 5 seconds apart, and requires `UP` with `CONNECTED`	Prove that the deployed application starts and reaches its database

Table 5.3: Stages of the Jenkins pipeline

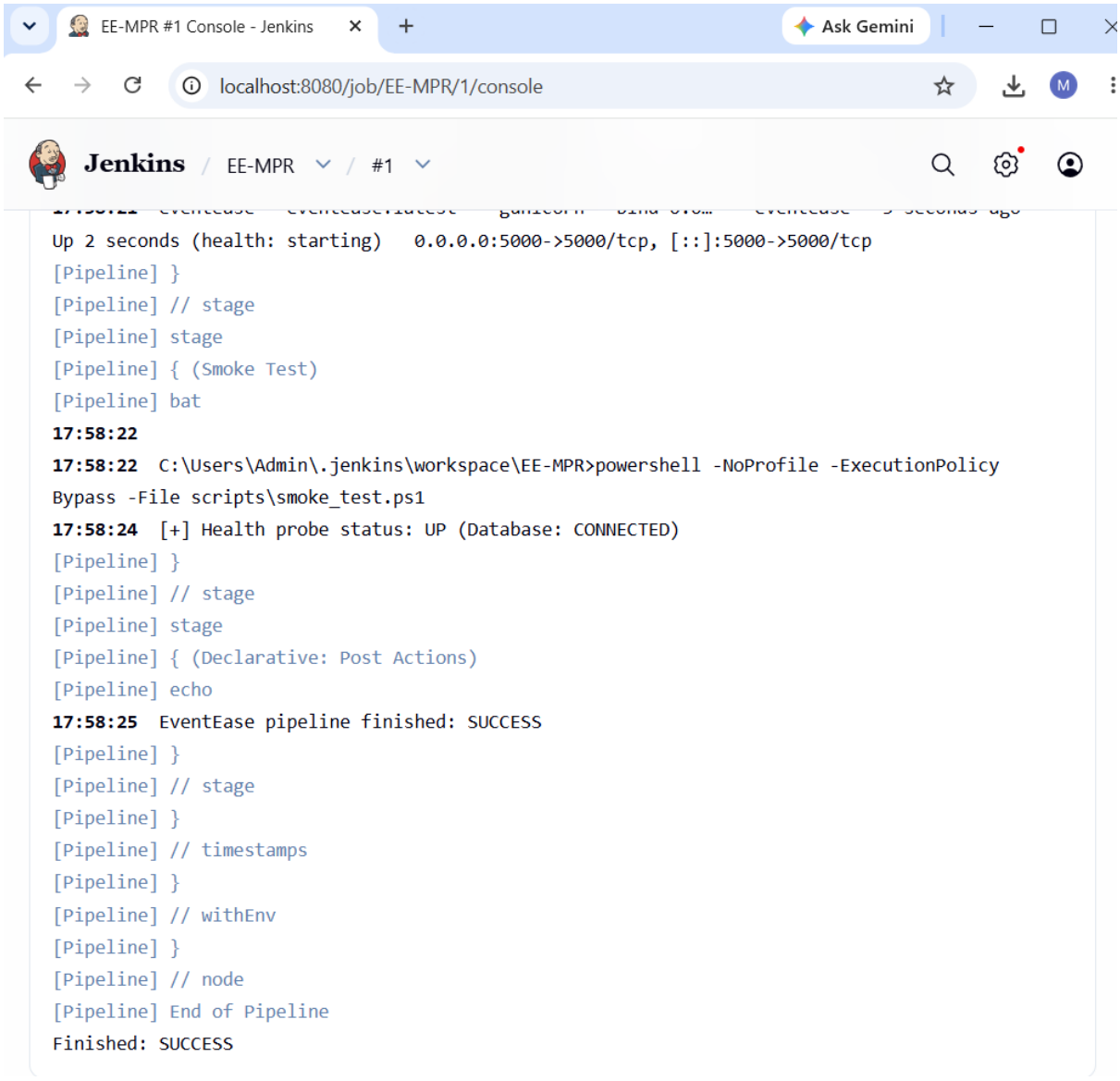


Figure 5.11: Jenkins console output of a successful pipeline run

The console output can be read from top to bottom as the story of one build. The build is job EE-MPR, run #1, and it ran in the Jenkins workspace C:\Users\Admin\.jenkins\workspace\EE-MPR. The Deploy stage ends with docker compose ps, which lists the eventease container on port 5000 as Up with its health check still starting. In the Smoke Test stage Jenkins runs scripts\smoke_test.ps1 through PowerShell and the log prints "[+] Health probe status: UP (Database: CONNECTED)". The post action then prints "EventEase pipeline finished: SUCCESS" and the console ends with "Finished: SUCCESS". If any test failed, the Unit Tests stage would fail and the later stages (build, deploy, smoke test) would not run, so a broken change is stopped before it is released.

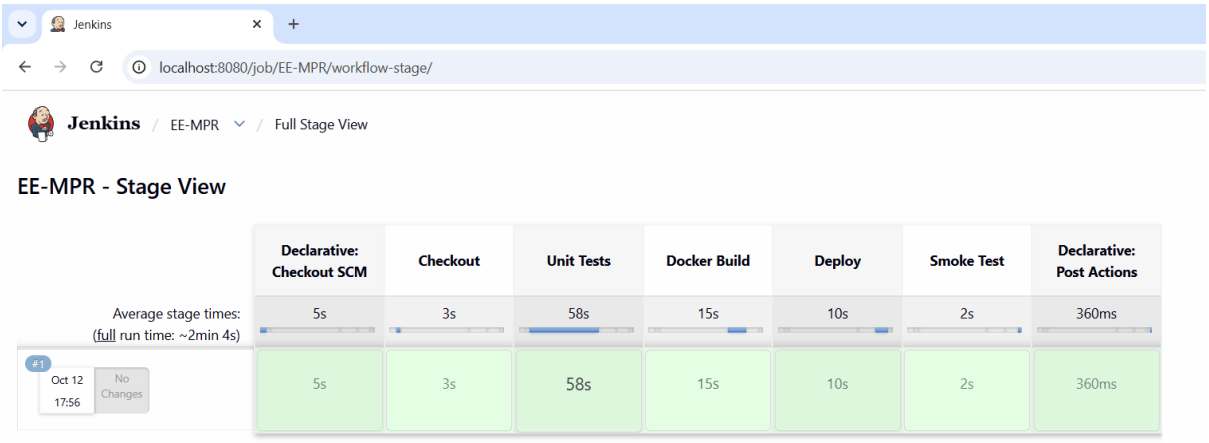


Figure 5.12: Jenkins stage view of the pipeline

The stage view shows run #1 of 12 October at 17:56 with every stage green. Besides the declarative steps Checkout SCM (5 s) and Post Actions (360 ms), the five stages took 3 s (Checkout), 58 s (Unit Tests), 15 s (Docker Build), 10 s (Deploy) and 2 s (Smoke Test), and the whole run took about 2 minutes. The smoke test is the most important check of the deployment: the unit tests prove that the code is correct in isolation, but a wrong port, a missing environment variable or an unreachable database would pass every unit test and still fail for a real user. Calling `/health` on the running container proves the deployed system works.

5.5 Monitoring with Nagios

Once the stack is deployed, it has to be watched. The Docker health check works from inside the container; Nagios Core works from outside, like a real user or an operator, and records the state of each service. Nagios is started from a second Compose file (`docker-compose.monitoring.yml`) and is available on port 8081. Its configuration is kept in the folder `nagios-config` and loaded into the container as read-only files.

Service	Check	What a failure means
Host `eventease-production-host`	TCP check of port 5000	The application is not listening
EventEase Web Portal - Port 5000	HTTP request to `/`, expects status 200	The home page cannot be served
Backend Health and Database - `health`	HTTP request to `/health`, expects 200 and the text `UP`	The application or its database is down
Events Page - database read	HTTP request to `/events`, expects 200	Data cannot be read from the database
Monitoring Container Disk Space	`check_local_disk` with warning 20 % and critical 10 % free	The disk is nearly full

Table 5.4: Nagios host and services

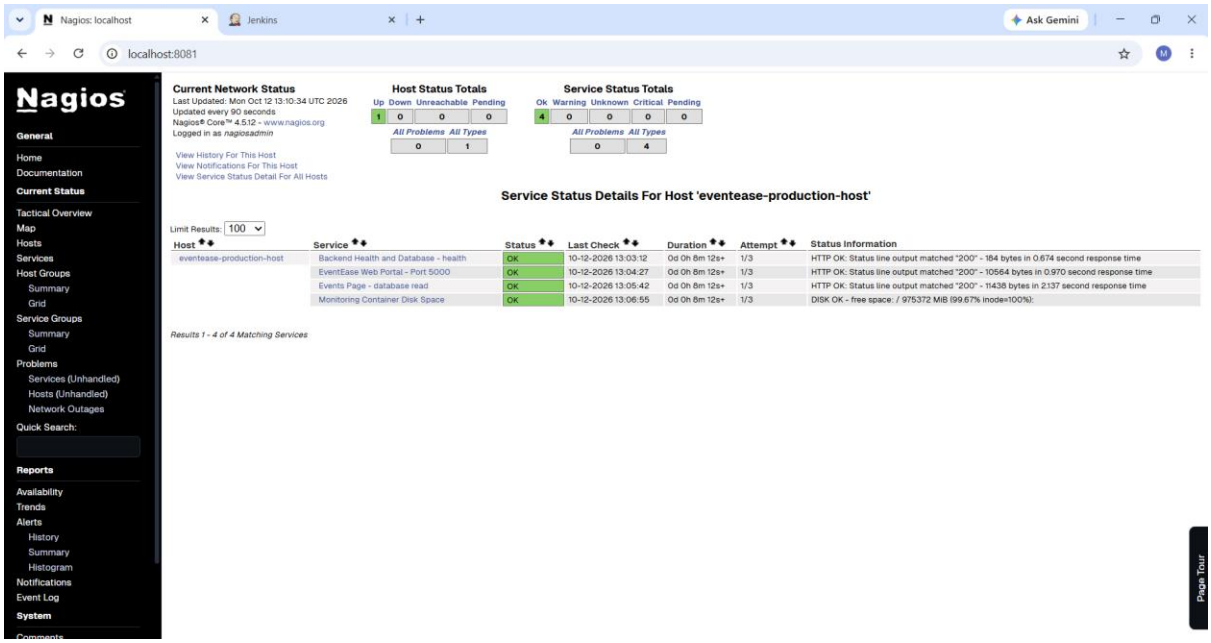


Figure 5.13: Nagios Service Status Details for eventease-production-host

The Services page lists the checks of the host `eventease-production-host`. The host is Up and all four services are OK, with Warning, Unknown, Critical and Pending all at 0. The health endpoint returns HTTP OK with the status line matched to 200 (184 bytes in 0.674 s), the web portal returns HTTP OK (10564 bytes in 0.970 s), the events page returns HTTP OK (11438 bytes in 2.137 s), and the disk check reports DISK OK with 975372 MiB (99.67 %) free. The columns Last Check, Duration and Attempt show that the checks are repeated at regular intervals and that Nagios retries a failed check before it reports a hard failure, to avoid false alarms.

5.6 How CI/CD Is Achieved End to End

Continuous integration and deployment in this project is a chain in which the output of each tool is the input of the next.

1. **Plan.** A story is created and assigned in Jira; the card in To Do is the request for change.
2. **Commit.** The team member works on a feature branch and commits with the Jira key in the message.
3. **Integrate.** The branch is pushed to GitHub and merged into `main` through a pull request; GitHub Actions runs the tests on the pull request.
4. **Build and test.** Jenkins checks out `main`, runs the pytest suite and records the JUnit report. A failing test stops the pipeline.
5. **Package.** The Docker image is built from the repository, so the application is delivered as the same artefact on every machine.
6. **Deploy.** Docker Compose starts the container with its volume and health check.
7. **Verify.** The smoke test calls `/health` and requires UP with the database CONNECTED.

8. **Monitor and close.** Nagios checks the portal, the health endpoint, the events page and the disk, and the story is moved to Done in Jira.

Stage	Tool	Output	Evidence
Plan	Jira	Story on the board	Figures 2.1 to 2.7
Source control	Git, GitHub	Commits, merged pull requests	Figures 5.2 to 5.6
Pre-merge test	GitHub Actions	Green check on the pull request	Figure 5.7
CI and CD	Jenkins, pytest	Tests passed, JUnit report, deployed stack	Figure 5.11, Figure 5.12
Packaging and run	Docker, Compose	Image and healthy container	Figures 5.8 to 5.10
Monitoring	Nagios	All services OK	Figure 5.13

Table 5.5: The CI/CD chain and the evidence for each stage

6. Testing

6.1 Automated Tests

The application is tested with pytest. The file `test_app.py` contains 21 tests that use Flask's test client and a fresh temporary database for every test, so the tests never touch the real `database.db`. They run in the Unit Tests stage of the Jenkins pipeline and in the GitHub Actions workflow, and Jenkins records the results from `test-reports/junit.xml`.

No.	Test	What is verified
1	`test_home_page`	Home page answers with status 200
2	`test_registration_page`	Registration page answers with status 200
3	`test_login_page`	Login page answers with status 200
4	`test_events_page_lists_college_events`	Events page shows the college events (Techathon, Shazam: Football)
5	`test_event_details_and_missing_event`	An existing event opens; an unknown event returns 404
6	`test_health_check_success`	`/health` reports UP and CONNECTED
7	`test_health_check_failure`	`/health` reports DOWN with status 503 when the database fails
8	`test_invalid_email_registration`	Registration with an invalid e-mail is rejected
9	`test_short_password_rejected`	A password shorter than eight characters is rejected
10	`test_successful_registration_then_duplicate_email`	Registration works once; the same e-mail again is rejected
11	`test_login_success_and_wrong_password`	Correct password logs in; a wrong one is refused
12	`test_booking_requires_login`	Booking without logging in redirects to login
13	`test_successful_booking_appears_in_my_bookings`	A booking is saved and listed on My Bookings
14	`test_booking_more_than_capacity_is_rejected`	Booking more tickets than the seats left is rejected
15	`test_invalid_ticket_count_is_rejected`	A ticket count outside 1 to 10 is rejected
16	`test_student_cannot_open_admin_dashboard`	A student is redirected away from `/admin`

No.	Test	What is verified
17	`test_admin_can_add_event`	An admin can add an event and see it listed
18	`test_event_posters_are_served`	Event posters are served from `static/images`
19	`test_photo_overrides_drawn_poster`	A photo with the event's name replaces the drawn poster
20	`test_sold_out_event_shows_message`	A sold-out event shows a message instead of the booking form
21	`test_old_database_is_upgraded`	An old database gets the newer columns on start-up

Table 6.1: Automated test cases

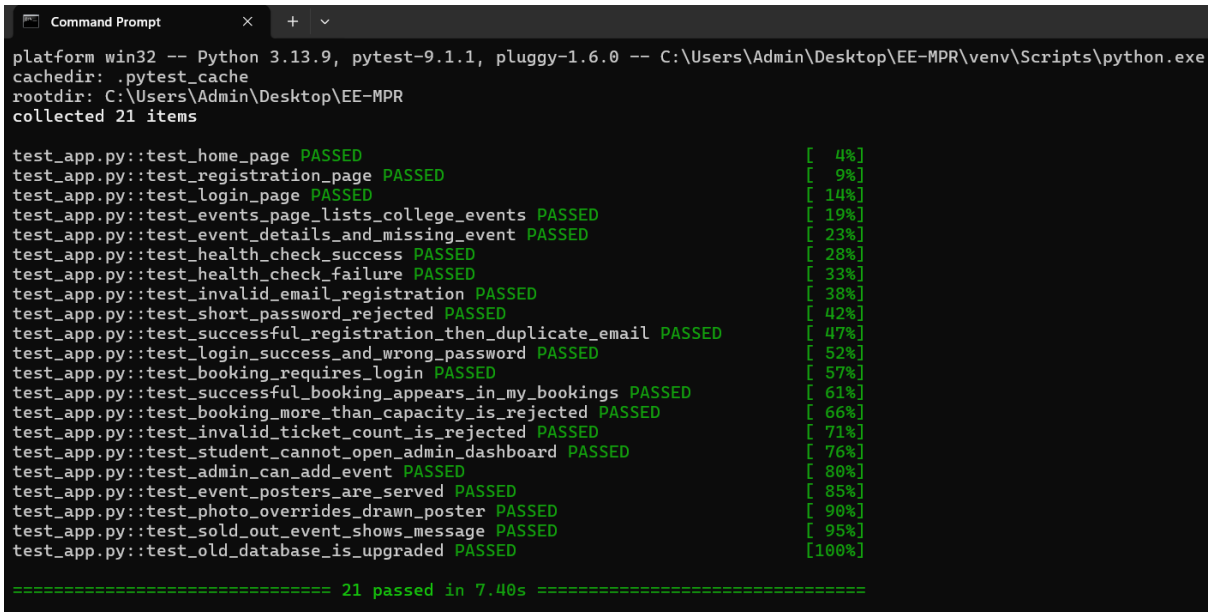


Figure 6.1: Output of python -m pytest -v

Measure	Value
Test framework	pytest 9.1.1 on Python 3.13.9
Tests collected	21
Tests passed	21 (100 %)
Duration	7.40 seconds
Report	`test-reports/junit.xml` (JUnit format)

Table 6.2: Result of the pytest run

6.2 Deployment Smoke Test

Unit tests prove the code; the smoke test proves the deployment. In the last stage of the pipeline `scripts/smoke_test.ps1` calls `http://localhost:5000/health` and the pipeline succeeds only when the answer is UP with the database CONNECTED. The same check was done by hand with `curl` (Figure 5.9).

6.3 Functional Verification

Check	Expected	Observed
Events on the home page	9 events, 6 clubs, 2 free events	9 events, 6 clubs and 2 free events, as shown in Figure 4.1
Category filter	Choosing Sports shows only the sports events	Four sports events (Football, Cricket, Badminton, Pickleball), as shown in Figure 4.2
Booking	A booking appears on My Bookings with the correct total	2 tickets of Techathon at ₹150 listed with a total of ₹300, as shown in Figure 4.5
Seats left	Capacity minus tickets booked	100 of 100 seats before booking, as shown in Figure 4.3, and 2 / 100 booked afterwards, as shown in Figure 4.6
Admin access	Only the admin sees the dashboard	The dashboard opens for the admin account, as shown in Figure 4.6
Health endpoint	`UP` and `CONNECTED`	UP and CONNECTED, as shown in Figure 5.9

Table 6.3: Cross-checks of the application

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Browse, search and filter events	Delivered: 9 events, search box and category filters (Figure 4.2)
Registration, login and booking	Delivered: validated registration, login, booking of 1 to 10 tickets with a capacity check (Figures 4.4, 4.3)
Personal bookings page	Delivered: My Bookings with totals (Figure 4.5)
Admin dashboard	Delivered: role-protected dashboard and Add Event form (Figure 4.6)
Agile planning in Jira	1 epic, 7 stories, 33 story points; two sprints completed (Figures 2.1 to 2.7)
Git workflow	Redesign and DevOps on separate branches, merged by pull requests with a green GitHub Actions check (Figures 5.3 to 5.7)
Containerised stack, one command	1 image, 1 volume, 1 network started by `docker compose up -d --build`; container healthy (Figures 5.8 to 5.10)
Jenkins pipeline	Checkout, 21 tests, Docker build, deploy and smoke test: Finished: SUCCESS (Figure 5.11)
Monitoring	Nagios: 1 host Up, 4 services OK (Figure 5.13)

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The project shows the practices working together. The redesign and the DevOps files lived on separate branches, so each could be reviewed and merged on its own through a pull request, and the Jira key in every commit message tied each change back to a story on the board. Once the code was on `main`, the pipeline gave immediate evidence about the combined result: the tests passed, the container started, the health probe answered UP, and Nagios confirmed from outside that the portal, the health endpoint and the events page were working. If any of these had failed, the cause would have been traced to a recent commit through its Jira key, fixed on a branch and delivered through the same route.

7.3 Challenges and Design Decisions

Challenge	Resolution
Tests must not damage real data	A `conftest.py` points the application to a temporary database before it is imported, and every test gets a fresh database
A running process is not necessarily a working service	A health endpoint that also queries the database, used by Docker, the smoke test and Nagios
Bookings must survive a rebuilt container	The SQLite file lives in the named volume `eventease-data`
Sample events must not be inserted twice	One Gunicorn worker with four threads, and events are inserted only when the table is empty
The database from the old version must keep working	The application adds the new columns on start-up and removes the old placeholder events

Challenge	Resolution
Secrets must not be in Git	`SECRET_KEY` is read from the environment; only `.env.example` is committed

Table 7.2: Challenges and how they were resolved

7.4 Limitations

- SQLite allows one writer at a time and the application runs one worker, which is enough for one college but would not scale to many simultaneous bookings.
- There is no online payment, ticket cancellation or e-mail confirmation; the total is calculated for information only.
- Jenkins runs on a development machine, so the pipeline is started by hand or by a periodic scan rather than by a GitHub webhook.
- The tests run on the host with Python 3.13 and in GitHub Actions with Python 3.13, while the image is based on Python 3.11.
- There is no automatic rollback to the previous image when a deployment fails.
- Jenkins, the application and Nagios share one machine, so a failure of that machine would stop both the system and the means of observing it.
- The application is served over plain HTTP and the default secret key is only suitable for development.

8. Conclusion and Future Scope

8.1 Conclusion

EventEase meets its functional goal: a student can find an event, register, book tickets that are checked against the seats left and see the booking on a personal page, while an administrator can add events and see how many seats are taken. Automated tests cover registration, login, booking, capacity limits, admin access and the health endpoint.

More importantly for this course, the project shows the lab experiments operating as one system. Planning the work in Jira gave each member a clear stream and a visible board. Git and GitHub kept the redesign and the DevOps files apart while they were in progress and together afterwards, with pull requests as the points of integration and a GitHub Actions check on each. Docker and Docker Compose turned the environment into files that are versioned with the code, so the stack that was tested was the stack that ran. The Jenkins pipeline checked out `main`, ran the tests, built the image, deployed it and ended with a smoke test that required the database to be connected. Nagios then watched the portal, the health endpoint, the events page and the disk. Each practice becomes more valuable because of the one that follows it.

8.2 Future Scope

- Add online payment, ticket cancellation and e-mail confirmation with a QR-code ticket.
- Let each club log in and manage only its own events.
- Move from SQLite to a client-server database and run several workers.
- Start the Jenkins pipeline automatically after every merge through a webhook, push tagged images to a registry and add automatic rollback.
- Measure code coverage, add automated browser tests, and send Nagios alerts to e-mail or a chat channel.
- Serve the application over HTTPS on a college server.

References

1. Flask documentation. https://flask.palletsprojects.com
2. Gunicorn documentation. https://docs.gunicorn.org
3. SQLite documentation. https://www.sqlite.org/docs.html
4. Pro Git, Scott Chacon and Ben Straub. https://git-scm.com/book
5. GitHub documentation: pull requests and GitHub Actions. https://docs.github.com
6. Docker documentation. https://docs.docker.com
7. Docker Compose documentation. https://docs.docker.com/compose
8. Jenkins User Documentation: Pipeline. https://www.jenkins.io/doc/book/pipeline
9. pytest documentation. https://docs.pytest.org
10. Atlassian Jira Software documentation. https://support.atlassian.com/jira-software-cloud
11. Nagios Core documentation. https://www.nagios.org/documentation
12. Agile Software Development & DevOps Lab manual, Thadomal Shahani Engineering College, 2026–2027.

Appendix A: Running the Project

A.1 Prerequisites

- Git, Docker Desktop (with Docker Compose) and Python 3 with `pip`.

A.2 Run locally

```
git clone repository URL
cd EventEase
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m pytest -v
python app.py
```

Open `http://127.0.0.1:5000` in a browser.

A.3 Run with Docker

```
docker compose up -d --build
docker compose ps
curl http://localhost:5000/health
```

The expected answer of the last command is `{"database":"CONNECTED","status":"UP"}`. To start Nagios: `docker compose -f docker-compose.monitoring.yml up -d`, then open `http://localhost:8081`.

A.4 Make an administrator

Register an account on the website, then run `python create_admin.py` and enter its e-mail address.

A.5 Addresses

Component	Address	Purpose
Web application	http://localhost:5000	Events, booking, admin
Health endpoint	http://localhost:5000/health	Service and database status
Jenkins	http://localhost:8080	Pipeline console
Nagios	http://localhost:8081	Service status

Table A.1: Addresses used by the project

A.6 Repository Layout

Path	Content
app.py, database.py	Flask application and database set-up
templates/, static/	Pages, stylesheet and event posters
test_app.py, conftest.py	pytest tests and the temporary-database fixture
Dockerfile, docker-compose.yml	Image and application stack
docker-compose.monitoring.yml, nagios-config/	Nagios stack and its configuration

Path	Content
Jenkinsfile, scripts/smoke_test.ps1	Pipeline definition and the smoke test
.github/workflows/python-tests.yml	GitHub Actions test workflow
create_admin.py	Grants the admin role to a registered user
.env.example, .gitignore, README.md	Settings template, ignore rules, project description

Table A.2: Layout of the repository